DATABASE DESIGN DOCUMENT

Virginia Market Square

Community Vendors Platform — 12-Table MySQL Schema

Course	CIS2987 — Web Development Capstone
Project	Virginia Market Square — Community Vendors Platform
Database	MySQL 8.0 / MariaDB
Normal Form	Third Normal Form (3NF)
Total Tables	12 tables across 4 functional groups
Total Fields	94 columns with appropriate data types
Version	1.0 — March 2026

1. Introduction

This document describes the complete relational database design for Virginia Market Square, a community farmers market platform built as a CIS2987 capstone project. The database powers all aspects of the website including vendor profiles, product listings, e-commerce transactions, user authentication, and administrative functions.

The schema is designed to Third Normal Form (3NF), eliminating data redundancy and ensuring update anomalies are prevented. All tables use InnoDB storage engine to support foreign key constraints and ACID-compliant transactions. Every monetary value is stored as DECIMAL(10,2) to avoid floating-point precision errors.

2. Design Principles & Conventions

2.1 Normalization

The schema follows Third Normal Form (3NF):

- 1NF: All columns hold atomic values. No repeating groups or arrays.
- 2NF: Every non-key column is fully functionally dependent on the entire primary key.
- 3NF: No transitive dependencies — non-key columns depend only on the primary key.
- Exception: order_items.vendor_id is intentionally denormalized for fast vendor sales queries without a JOIN through products.

2.2 Naming Conventions

Convention	Rule	Example
Table names	Lowercase snake_case, plural	order_items, vendor_applications
Column names	Lowercase snake_case	created_date, business_name
Primary keys	table_name_id pattern	user_id, vendor_id, product_id
Foreign keys	Same name as referenced PK	vendor_id references vendors.vendor_id
ENUM values	Lowercase with underscores	market_day, special_event
Boolean flags	Prefix: is_ or has_	is_available, is_active
Date columns	_date suffix for DATE, _date for DATETIME	order_date, created_date
Index names	idx_ prefix or uk_ for unique	idx_vendor_id, uk_email

2.3 Data Type Standards

Data Category	MySQL Type	Rationale
Primary / Foreign Keys	INT AUTO_INCREMENT	Efficient integer joins; auto-increment avoids manual ID assignment
Money / Prices	DECIMAL(10,2)	Exact decimal arithmetic — FLOAT causes rounding errors in financial data
Short strings	VARCHAR(n)	Variable-length; n chosen to match real-world max (255 for names/URLs)
Long content	TEXT	Descriptions, messages, addresses — no length limit needed
Fixed-set values	ENUM	Enforces valid values at DB level; faster than VARCHAR checks
Date + Time	DATETIME	Stores full timestamp; use DATE-only where time is irrelevant (event_date)
True/False flags	BOOLEAN	Stored as TINYINT(1) in MySQL; readable in PHP as true/false
Distance	DECIMAL(5,2)	Enough for 999.99 miles with two decimal places

3. Schema Overview

The 12 tables are organized into four functional groups. Each group is color-coded consistently throughout this document and the ER diagram.

Group	Tables	Purpose
Authentication & Users	users, vendors, customers	Login credentials, role routing, vendor profiles, customer addresses
Product Catalog	categories, products	Category lookup and full product listings with inventory tracking
E-Commerce	cart, orders, order_items, transactions	Shopping cart, order lifecycle, line items, and payment records
Support	events, contacts, vendor_applications	Market calendar, public contact form submissions, vendor applications

3.1 Entity Relationships Summary

Parent Table	Cardinality	Child Table	FK Column
users	1 : N	vendors	vendors.user_id
users	1 : 1	customers	customers.user_id
vendors	1 : N	products	products.vendor_id
categories	1 : N	products	products.category_id
customers	1 : N	cart	cart.customer_id
products	1 : N	cart	cart.product_id
customers	1 : N	orders	orders.customer_id
orders	1 : N	order_items	order_items.order_id
products	1 : N	order_items	order_items.product_id
vendors	1 : N	order_items	order_items.vendor_id
orders	1 : N	transactions	transactions.order_id
vendors	1 : N	contacts (optional)	contacts.vendor_id

4. Table Definitions

Group: Authentication & Users

4.1 Table: `users`

Central authentication table. Stores login credentials and user type for all three role types: admin, vendor, and customer. Password hashing is handled server-side with PHP password_hash() using bcrypt before insertion.

Column	Data Type	Null?	Key	Default / Extra	Notes
user_id	INT	NOT NULL	PK	AUTO_INCREMENT	Unique identifier for every user
email	VARCHAR(255)	NOT NULL	UK		Login identifier — enforced unique
password_hash	VARCHAR(255)	NOT NULL			bcrypt hash, never plain text
full_name	VARCHAR(255)	NULL			Display name
user_type	ENUM	NOT NULL		admin|vendor|customer	Controls login routing and permissions
created_date	DATETIME	NOT NULL		DEFAULT NOW()	Account creation timestamp
last_login	DATETIME	NULL			Updated on each successful login
is_active	BOOLEAN	NOT NULL		DEFAULT 1	Soft-delete / account suspension flag

Indexes & Constraints:

- PRIMARY KEY (user_id)
- UNIQUE KEY uk_email (email)
- INDEX idx_user_type (user_type)

Relationships:

- users.user_id → vendors.user_id (1:N)
- users.user_id → customers.user_id (1:1)

CREATE TABLE Statement:

```
CREATE TABLE users (
  user_id       INT           NOT NULL AUTO_INCREMENT,
  email         VARCHAR(255)  NOT NULL,
  password_hash VARCHAR(255)  NOT NULL,
  full_name     VARCHAR(255)  NULL,
  user_type     ENUM('admin','vendor','customer') NOT NULL,
  created_date  DATETIME      NOT NULL DEFAULT NOW(),
  last_login    DATETIME      NULL,
  is_active     BOOLEAN       NOT NULL DEFAULT 1,
  PRIMARY KEY (user_id),
  UNIQUE KEY uk_email (email),
  INDEX idx_user_type (user_type)
);
```

4.2 Table: vendors

Stores vendor business profiles. Linked to users via user_id so vendors have both login credentials (in users) and business information here. The verified flag is set by admin after reviewing the application.

Column	Data Type	Null?	Key	Default / Extra	Notes

vendor_id	INT	NOT NULL	**PK**	AUTO_INCREMENT	Unique vendor identifier
user_id	INT	NOT NULL	**FK**	→ users	Links to login credentials
business_name	VARCHAR(255)	NOT NULL			Displayed on vendor directory
description	TEXT	NULL			Vendor bio / about section
category	VARCHAR(100)	NULL			e.g. Produce, Bakery, Crafts
phone	VARCHAR(20)	NULL			Optional contact phone
website	VARCHAR(255)	NULL			Optional website URL
image_url	VARCHAR(255)	NULL			Vendor logo / profile photo
verified	BOOLEAN	NOT NULL		DEFAULT 0	Set TRUE by admin after approval
verified_date	DATETIME	NULL			Timestamp of verification
miles_from_virginia	DECIMAL(5,2)	NULL			Must be ≤50 miles — policy requirement
created_date	DATETIME	NOT NULL		DEFAULT NOW()	Profile creation date

Indexes & Constraints:

- `PRIMARY KEY (vendor_id)`
- `UNIQUE KEY uk_vendor_user (user_id)`
- `INDEX idx_verified (verified)`
- `INDEX idx_category (category)`

Relationships:

- vendors.user_id → users.user_id (FK)
- vendors.vendor_id → products.vendor_id (1:N)
- vendors.vendor_id → order_items.vendor_id (1:N)
- vendors.vendor_id → contacts.vendor_id (1:N optional)

CREATE TABLE Statement:

```
CREATE TABLE vendors (
  vendor_id           INT           NOT NULL AUTO_INCREMENT,
  user_id             INT           NOT NULL,
  business_name       VARCHAR(255)  NOT NULL,
  description         TEXT          NULL,
  category            VARCHAR(100)  NULL,
  phone               VARCHAR(20)   NULL,
  website             VARCHAR(255)  NULL,
  image_url           VARCHAR(255)  NULL,
  verified            BOOLEAN       NOT NULL DEFAULT 0,
  verified_date       DATETIME      NULL,
  miles_from_virginia DECIMAL(5,2)  NULL,
  created_date        DATETIME      NOT NULL DEFAULT NOW(),
  PRIMARY KEY (vendor_id),
  UNIQUE KEY uk_vendor_user (user_id),
```

```
  INDEX idx_verified (verified),
  FOREIGN KEY (user_id) REFERENCES users(user_id) ON DELETE CASCADE
);
```

4.3 Table: customers

Stores customer shipping addresses and contact details. Has a 1:1 relationship with users — every customer has one users record, and a users record can have at most one customers record. Shipping address is stored here for checkout pre-fill.

Column	Data Type	Null?	Key	Default / Extra	Notes
customer_id	INT	NOT NULL	PK	AUTO_INCREMENT	Unique customer identifier
user_id	INT	NOT NULL	FK	→ users	Links to login credentials (1:1)
phone	VARCHAR(20)	NULL			Optional contact number
address_street	VARCHAR(255)	NULL			Default shipping street
address_city	VARCHAR(100)	NULL			Default shipping city
address_state	VARCHAR(2)	NULL			2-char state code e.g. MN
address_zip	VARCHAR(10)	NULL			ZIP or ZIP+4
created_date	DATETIME	NOT NULL		DEFAULT NOW()	Customer account creation date

Indexes & Constraints:

- PRIMARY KEY (customer_id)
- UNIQUE KEY uk_customer_user (user_id)

Relationships:

- customers.user_id → users.user_id (FK)
- customers.customer_id → orders.customer_id (1:N)
- customers.customer_id → cart.customer_id (1:N)

CREATE TABLE Statement:

```
CREATE TABLE customers (
  customer_id    INT           NOT NULL AUTO_INCREMENT,
  user_id        INT           NOT NULL,
  phone          VARCHAR(20)   NULL,
  address_street VARCHAR(255)  NULL,
  address_city   VARCHAR(100)  NULL,
  address_state  VARCHAR(2)    NULL,
  address_zip    VARCHAR(10)   NULL,
  created_date   DATETIME      NOT NULL DEFAULT NOW(),
  PRIMARY KEY (customer_id),
  UNIQUE KEY uk_customer_user (user_id),
  FOREIGN KEY (user_id) REFERENCES users(user_id) ON DELETE CASCADE
);
```

Group: Product Catalog

4.4 Table: categories

Lookup / reference table for product categories (e.g. Produce, Bakery, Dairy, Crafts). Keeps category names normalized so they can be renamed without updating every products row. display_order controls sort order in navigation menus.

Column	Data Type	Null?	Key	Default / Extra	Notes
category_id	INT	NOT NULL	PK	AUTO_INCREMENT	Unique category identifier
category_name	VARCHAR(100)	NOT NULL	UK		Enforced unique — e.g. Produce
description	TEXT	NULL			Optional description for display
display_order	INT	NOT NULL		DEFAULT 0	Lower = appears first in menus

Indexes & Constraints:

- PRIMARY KEY (category_id)
- UNIQUE KEY uk_category_name (category_name)
- INDEX idx_display_order (display_order)

Relationships:

- categories.category_id → products.category_id (1:N)

CREATE TABLE Statement:

```
CREATE TABLE categories (
  category_id   INT          NOT NULL AUTO_INCREMENT,
  category_name VARCHAR(100) NOT NULL,
  description   TEXT         NULL,
  display_order INT          NOT NULL DEFAULT 0,
  PRIMARY KEY (category_id),
  UNIQUE KEY uk_category_name (category_name),
  INDEX idx_display_order (display_order)
);
```

4.5 Table: products

Core product listing table. Each product belongs to exactly one vendor and one category. is_available acts as a soft visibility toggle — vendors can hide items without deleting them. unit_price is the current price; historical price is snapshotted in order_items at time of purchase.

Column	Data Type	Null?	Key	Default / Extra	Notes
product_id	INT	NOT NULL	PK	AUTO_INCREMENT	Unique product identifier
vendor_id	INT	NOT NULL	FK	→ vendors	Product owner
category_id	INT	NULL	FK	→ categories	NULL if uncategorized

product_name	VARCHAR(255)	NOT NULL			Display name
description	TEXT	NULL			Full product description
price	DECIMAL(10,2)	NULL			Current unit price in USD
stock_quantity	INT	NOT NULL		DEFAULT 0	Current available inventory
image_url	VARCHAR(255)	NULL			Product photo path
is_available	BOOLEAN	NOT NULL		DEFAULT 1	Visibility toggle for vendor use
created_date	DATETIME	NOT NULL		DEFAULT NOW()	Listing creation date
updated_date	DATETIME	NULL		ON UPDATE NOW()	Auto-updated on any change

Indexes & Constraints:

- PRIMARY KEY (product_id)
- INDEX idx_vendor_id (vendor_id)
- INDEX idx_category_id (category_id)
- INDEX idx_is_available (is_available)
- INDEX idx_price (price)

Relationships:

- products.vendor_id → vendors.vendor_id (FK ON DELETE CASCADE)
- products.category_id → categories.category_id (FK)
- products.product_id → cart.product_id (1:N)
- products.product_id → order_items.product_id (1:N)

CREATE TABLE Statement:

```
CREATE TABLE products (
  product_id     INT            NOT NULL AUTO_INCREMENT,
  vendor_id      INT            NOT NULL,
  category_id    INT            NULL,
  product_name   VARCHAR(255)   NOT NULL,
  description    TEXT           NULL,
  price          DECIMAL(10,2)  NULL,
  stock_quantity INT            NOT NULL DEFAULT 0,
  image_url      VARCHAR(255)   NULL,
  is_available   BOOLEAN        NOT NULL DEFAULT 1,
  created_date   DATETIME       NOT NULL DEFAULT NOW(),
  updated_date   DATETIME       NULL ON UPDATE NOW(),
  PRIMARY KEY (product_id),
  INDEX idx_vendor_id (vendor_id),
  INDEX idx_category_id (category_id),
  FOREIGN KEY (vendor_id)   REFERENCES vendors(vendor_id)    ON DELETE CASCADE,
  FOREIGN KEY (category_id) REFERENCES categories(category_id) ON DELETE SET NULL
);
```

Group: E-Commerce

4.6 Table: cart

Persists shopping cart contents in the database (not PHP sessions). This allows carts to survive browser restarts and session timeouts. A composite unique key on (customer_id, product_id) prevents duplicate cart rows — adding the same item twice increments quantity instead.

Column	Data Type	Null?	Key	Default / Extra	Notes
cart_id	INT	NOT NULL	**PK**	AUTO_INCREMENT	Unique cart line identifier
customer_id	INT	NOT NULL	**FK**	→ customers	Cart owner
product_id	INT	NOT NULL	**FK**	→ products	Item in cart
quantity	INT	NOT NULL		DEFAULT 1	Must be ≥ 1
added_date	DATETIME	NOT NULL		DEFAULT NOW()	When item was added

Indexes & Constraints:

- PRIMARY KEY (cart_id)
- UNIQUE KEY uk_cart_item (customer_id, product_id)
- INDEX idx_cart_customer (customer_id)

Relationships:

- cart.customer_id → customers.customer_id (FK ON DELETE CASCADE)
- cart.product_id → products.product_id (FK ON DELETE CASCADE)

CREATE TABLE Statement:

```
CREATE TABLE cart (
  cart_id     INT      NOT NULL AUTO_INCREMENT,
  customer_id INT      NOT NULL,
  product_id  INT      NOT NULL,
  quantity    INT      NOT NULL DEFAULT 1,
  added_date  DATETIME NOT NULL DEFAULT NOW(),
  PRIMARY KEY (cart_id),
  UNIQUE KEY uk_cart_item (customer_id, product_id),
  INDEX idx_cart_customer (customer_id),
  FOREIGN KEY (customer_id) REFERENCES customers(customer_id) ON DELETE CASCADE,
  FOREIGN KEY (product_id)  REFERENCES products(product_id)   ON DELETE CASCADE
);
```

4.7 Table: orders

Master order header record. Created when a customer completes checkout. Stores total amounts, order status, payment status, and the shipping address snapshot (copied from customers at checkout time — shipping address may change later). Line items are stored separately in order_items.

Column	Data Type	Null?	Key	Default / Extra	Notes
order_id	INT	NOT NULL	**PK**	AUTO_INCREMENT	Unique order number

customer_id	INT	NOT NULL	**FK**	→ customers	Order owner
order_date	DATETIME	NOT NULL		DEFAULT NOW()	Timestamp when order was placed
total_amount	DECIMAL(10,2)	NULL			subtotal + tax + shipping
subtotal	DECIMAL(10,2)	NULL			Sum of all line_total values
tax	DECIMAL(10,2)	NOT NULL		DEFAULT 0.00	Calculated sales tax
status	ENUM	NOT NULL		pending|confirmed|shipped|delivered|cancelled	Fulfillment status
payment_status	ENUM	NOT NULL		unpaid|paid|refunded	Updated by transactions table
shipping_address	TEXT	NULL			Address snapshot at checkout
notes	TEXT	NULL			Customer delivery instructions

Indexes & Constraints:

- `PRIMARY KEY (order_id)`
- `INDEX idx_customer_id (customer_id)`
- `INDEX idx_status (status)`
- `INDEX idx_order_date (order_date)`

Relationships:

- orders.customer_id → customers.customer_id (FK)
- orders.order_id → order_items.order_id (1:N)
- orders.order_id → transactions.order_id (1:N)

CREATE TABLE Statement:

```
CREATE TABLE orders (
  order_id         INT            NOT NULL AUTO_INCREMENT,
  customer_id      INT            NOT NULL,
  order_date       DATETIME       NOT NULL DEFAULT NOW(),
  total_amount     DECIMAL(10,2)  NULL,
  subtotal         DECIMAL(10,2)  NULL,
  tax              DECIMAL(10,2)  NOT NULL DEFAULT 0.00,
  status           ENUM('pending','confirmed','shipped',
                        'delivered','cancelled') NOT NULL DEFAULT 'pending',
  payment_status   ENUM('unpaid','paid','refunded') NOT NULL DEFAULT 'unpaid',
  shipping_address TEXT           NULL,
  notes            TEXT           NULL,
  PRIMARY KEY (order_id),
  INDEX idx_customer_id (customer_id),
  INDEX idx_status (status),
  FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
```

```
);
```

4.8 Table: order_items

Stores individual line items for each order. This is the junction table between orders and products. unit_price is a snapshot of the price at the time of purchase — this preserves order history even if the vendor later changes the product price. vendor_id is also denormalized here for fast vendor-specific sales queries without an extra JOIN through products.

Column	Data Type	Null?	Key	Default / Extra	Notes
order_item_id	INT	NOT NULL	PK	AUTO_INCREMENT	Unique line item identifier
order_id	INT	NOT NULL	FK	→ orders	Parent order
product_id	INT	NOT NULL	FK	→ products	Product reference
vendor_id	INT	NOT NULL	FK	→ vendors	Denormalized for fast vendor queries
quantity	INT	NOT NULL			Quantity ordered
unit_price	DECIMAL(10,2)	NOT NULL			Price snapshot at purchase time
line_total	DECIMAL(10,2)	NOT NULL			quantity × unit_price

Indexes & Constraints:

- PRIMARY KEY (order_item_id)
- INDEX idx_order_id (order_id)
- INDEX idx_product_id (product_id)
- INDEX idx_vendor_id (vendor_id)

Relationships:

- order_items.order_id → orders.order_id (FK ON DELETE CASCADE)
- order_items.product_id → products.product_id (FK)
- order_items.vendor_id → vendors.vendor_id (FK)

CREATE TABLE Statement:

```
CREATE TABLE order_items (
  order_item_id INT            NOT NULL AUTO_INCREMENT,
  order_id      INT            NOT NULL,
  product_id    INT            NOT NULL,
  vendor_id     INT            NOT NULL,
  quantity      INT            NOT NULL,
  unit_price    DECIMAL(10,2)  NOT NULL,
  line_total    DECIMAL(10,2)  NOT NULL,
  PRIMARY KEY (order_item_id),
  INDEX idx_order_id   (order_id),
  INDEX idx_product_id (product_id),
  INDEX idx_vendor_id  (vendor_id),
  FOREIGN KEY (order_id)   REFERENCES orders(order_id)   ON DELETE CASCADE,
  FOREIGN KEY (product_id) REFERENCES products(product_id),
```

```
  FOREIGN KEY (vendor_id)  REFERENCES vendors(vendor_id)
);
```

4.9 Table: transactions

Records each payment attempt for an order. An order may have multiple transaction records if an initial attempt fails and the customer retries. The payment_gateway_id stores the Stripe charge ID (e.g. ch_3Nxyz...) for audit and refund purposes. transaction_status is updated via the Stripe webhook.

Column	Data Type	Null?	Key	Default / Extra	Notes
transaction_id	INT	NOT NULL	**PK**	AUTO_INCREMENT	Unique transaction record
order_id	INT	NOT NULL	**FK**	→ orders	Associated order
payment_method	ENUM	NOT NULL		credit_card|debit_card|paypal|stripe	Payment type used
payment_gateway_id	VARCHAR(255)	NULL			Stripe charge ID for audit trail
amount	DECIMAL(10,2)	NOT NULL			Amount charged / attempted
transaction_status	ENUM	NOT NULL		pending|success|failed|refunded	Updated via Stripe webhook
error_message	TEXT	NULL			Populated on failed attempts
transaction_date	DATETIME	NOT NULL		DEFAULT NOW()	Attempt timestamp

Indexes & Constraints:

- PRIMARY KEY (transaction_id)
- INDEX idx_order_id (order_id)
- INDEX idx_transaction_status (transaction_status)
- INDEX idx_gateway_id (payment_gateway_id)

Relationships:

- transactions.order_id → orders.order_id (FK)

CREATE TABLE Statement:

```
CREATE TABLE transactions (
  transaction_id     INT            NOT NULL AUTO_INCREMENT,
  order_id           INT            NOT NULL,
  payment_method     ENUM('credit_card','debit_card','paypal','stripe') NOT NULL,
  payment_gateway_id VARCHAR(255)   NULL,
  amount             DECIMAL(10,2)  NOT NULL,
  transaction_status ENUM('pending','success','failed','refunded')
```

```
                    NOT NULL DEFAULT 'pending',
  error_message      TEXT           NULL,
  transaction_date   DATETIME       NOT NULL DEFAULT NOW(),
  PRIMARY KEY (transaction_id),
  INDEX idx_order_id   (order_id),
  INDEX idx_gateway_id (payment_gateway_id),
  FOREIGN KEY (order_id) REFERENCES orders(order_id)
);
```

Group: Support

4.10 Table: events

Stores market events displayed on the public events calendar. No foreign keys — events are standalone content managed entirely by the admin. The event_type enum drives color-coding and filtering on the front end. Standalone table with no dependencies, so it can be truncated and repopulated freely.

Column	Data Type	Null?	Key	Default / Extra	Notes
event_id	INT	NOT NULL	**PK**	AUTO_INCREMENT	Unique event identifier
event_name	VARCHAR(255)	NOT NULL			Display title
event_date	DATE	NOT NULL			Date of the market event
event_time	TIME	NULL			Start time e.g. 14:30:00
description	TEXT	NULL			Full event description
event_type	ENUM	NOT NULL		market_day|special_event|workshop|promotion	Controls calendar color coding
created_date	DATETIME	NOT NULL		DEFAULT NOW()	Record creation date

Indexes & Constraints:

- PRIMARY KEY (event_id)
- INDEX idx_event_date (event_date)
- INDEX idx_event_type (event_type)

Relationships:

- No foreign key relationships — standalone content table

CREATE TABLE Statement:

```
CREATE TABLE events (
```

```
  event_id    INT          NOT NULL AUTO_INCREMENT,
  event_name  VARCHAR(255) NOT NULL,
  event_date  DATE         NOT NULL,
  event_time  TIME         NULL,
  description TEXT         NULL,
  event_type  ENUM('market_day','special_event','workshop','promotion')
              NOT NULL DEFAULT 'market_day',
  created_date DATETIME    NOT NULL DEFAULT NOW(),
  PRIMARY KEY (event_id),
  INDEX idx_event_date (event_date),
  INDEX idx_event_type (event_type)
);
```

4.11 Table: contacts

Captures public contact form submissions. vendor_id is an optional FK — when a visitor contacts a specific vendor, their message is linked here for admin routing. The status ENUM tracks the admin workflow: new → read → replied → archived.

Column	Data Type	Null?	Key	Default / Extra	Notes
contact_id	INT	NOT NULL	**PK**	AUTO_INCREMENT	Unique submission identifier
name	VARCHAR(255)	NOT NULL			Sender name
email	VARCHAR(255)	NOT NULL			Reply-to email address
phone	VARCHAR(20)	NULL			Optional phone
message	TEXT	NOT NULL			Message body
vendor_id	INT	NULL	**FK**	→ vendors (optional)	NULL = general inquiry
subject	VARCHAR(255)	NULL			Subject line from dropdown
status	ENUM	NOT NULL		new|read|replied|archived	Admin workflow status
submitted_date	DATETIME	NOT NULL		DEFAULT NOW()	Submission timestamp

Indexes & Constraints:

- PRIMARY KEY (contact_id)
- INDEX idx_vendor_id (vendor_id)
- INDEX idx_status (status)
- INDEX idx_submitted_date (submitted_date)

Relationships:

- contacts.vendor_id → vendors.vendor_id (FK SET NULL on delete — optional)

CREATE TABLE Statement:

```
CREATE TABLE contacts (
  contact_id     INT          NOT NULL AUTO_INCREMENT,
```

```
  name           VARCHAR(255) NOT NULL,
  email          VARCHAR(255) NOT NULL,
  phone          VARCHAR(20)  NULL,
  message        TEXT         NOT NULL,
  vendor_id      INT          NULL,
  subject        VARCHAR(255) NULL,
  status         ENUM('new','read','replied','archived')
                 NOT NULL DEFAULT 'new',
  submitted_date DATETIME     NOT NULL DEFAULT NOW(),
  PRIMARY KEY (contact_id),
  INDEX idx_vendor_id (vendor_id),
  INDEX idx_status    (status),
  FOREIGN KEY (vendor_id) REFERENCES vendors(vendor_id) ON DELETE SET NULL
);
```

4.12 Table: vendor_applications

Stores incoming vendor applications from the public application form. Intentionally has no FK to users — applications come from unauthenticated visitors who do not yet have an account. When admin approves an application, a separate process creates a users + vendors record. The admin_notes field is for internal reviewer comments.

Column	Data Type	Null?	Key	Default / Extra	Notes
application_id	INT	NOT NULL	**PK**	AUTO_INCREMENT	Unique application record
applicant_name	VARCHAR(255)	NOT NULL			Applicant full name
applicant_email	VARCHAR(255)	NOT NULL			Contact email for decision notice
applicant_phone	VARCHAR(20)	NULL			Optional phone
business_name	VARCHAR(255)	NOT NULL			Proposed vendor business name
business_description	TEXT	NULL			Products, methods, background
business_category	VARCHAR(100)	NULL			Requested category
miles_from_virginia	DECIMAL(5,2)	NULL			Self-reported — verified by admin
application_status	ENUM	NOT NULL		pending|approved|rejected	Admin decision field
admin_notes	TEXT	NULL			Internal reviewer notes
submitted_date	DATETIME	NOT NULL		DEFAULT NOW()	Application submission time
reviewed_date	DATETIME	NULL			Set when admin takes action

Indexes & Constraints:

- PRIMARY KEY (application_id)
- INDEX idx_application_status (application_status)

- `INDEX idx_submitted_date (submitted_date)`

Relationships:

- No FK relationships — pre-account submissions from anonymous applicants

CREATE TABLE Statement:

```
CREATE TABLE vendor_applications (
  application_id       INT           NOT NULL AUTO_INCREMENT,
  applicant_name       VARCHAR(255)  NOT NULL,
  applicant_email      VARCHAR(255)  NOT NULL,
  applicant_phone      VARCHAR(20)   NULL,
  business_name        VARCHAR(255)  NOT NULL,
  business_description TEXT          NULL,
  business_category    VARCHAR(100)  NULL,
  miles_from_virginia  DECIMAL(5,2)  NULL,
  application_status   ENUM('pending','approved','rejected')
                       NOT NULL DEFAULT 'pending',
  admin_notes          TEXT          NULL,
  submitted_date       DATETIME      NOT NULL DEFAULT NOW(),
  reviewed_date        DATETIME      NULL,
  PRIMARY KEY (application_id),
  INDEX idx_application_status (application_status),
  INDEX idx_submitted_date     (submitted_date)
);
```

5. Key SQL Queries

The following queries demonstrate the JOIN complexity the application relies on. All queries use prepared statements in PHP to prevent SQL injection.

1. Product catalog with vendor and category (JOIN)

```
SELECT p.product_id, p.product_name, p.price, p.stock_quantity,
       v.business_name AS vendor_name,
       c.category_name
FROM   products p
JOIN   vendors    v ON p.vendor_id   = v.vendor_id
LEFT JOIN categories c ON p.category_id = c.category_id
WHERE  p.is_available = 1
  AND  v.verified     = 1
ORDER  BY c.display_order, p.product_name;
```

2. Customer order history with line items

```
SELECT o.order_id, o.order_date, o.status, o.total_amount,
       p.product_name, oi.quantity, oi.unit_price, oi.line_total,
       v.business_name AS vendor
FROM   orders o
JOIN   order_items oi ON o.order_id    = oi.order_id
JOIN   products    p  ON oi.product_id = p.product_id
JOIN   vendors     v  ON oi.vendor_id  = v.vendor_id
WHERE  o.customer_id = :customer_id
ORDER  BY o.order_date DESC;
```

3. Vendor sales dashboard — revenue and units this month

```
SELECT p.product_name,
       SUM(oi.quantity)   AS units_sold,
       SUM(oi.line_total) AS revenue,
       COUNT(DISTINCT oi.order_id) AS order_count
FROM   order_items oi
JOIN   products p ON oi.product_id = p.product_id
JOIN   orders   o ON oi.order_id   = o.order_id
WHERE  oi.vendor_id = :vendor_id
  AND  MONTH(o.order_date) = MONTH(CURDATE())
  AND  YEAR(o.order_date)  = YEAR(CURDATE())
GROUP  BY p.product_id
ORDER  BY revenue DESC;
```

4. Shopping cart contents with current prices

```
SELECT c.cart_id, p.product_name, p.price AS current_price,
       c.quantity, (p.price * c.quantity) AS subtotal,
       v.business_name AS vendor, p.stock_quantity
FROM   cart c
JOIN   products p ON c.product_id = p.product_id
JOIN   vendors  v ON p.vendor_id  = v.vendor_id
WHERE  c.customer_id = :customer_id
```

```
  AND  p.is_available = 1;
```

5. Admin — all pending vendor applications

```
SELECT application_id, applicant_name, applicant_email,
       business_name, business_category,
       miles_from_virginia, submitted_date
FROM   vendor_applications
WHERE  application_status = 'pending'
ORDER  BY submitted_date ASC;
```

6. Database Security Measures

Security Measure	Implementation
Prepared Statements	All SQL queries use PDO prepared statements with bound parameters. No string concatenation into queries.
Password Hashing	Passwords stored using PHP password_hash() with PASSWORD_BCRYPT algorithm. password_verify() for authentication.
CSRF Protection	All HTML forms include a CSRF token stored in the PHP session. Token validated server-side before processing.
Input Validation	Server-side validation on all form inputs before any database write. htmlspecialchars() on all output.
Least Privilege	Application DB user granted only SELECT, INSERT, UPDATE, DELETE. No CREATE/DROP/ALTER permissions.
Foreign Keys	All FKs enforced with InnoDB constraints. Cascading deletes prevent orphaned records.
Session Security	PHP session_regenerate_id(true) on login. Session timeout after 30 minutes of inactivity.
SSL / HTTPS	Production site served over HTTPS. Database connection uses SSL where host supports it.

7. Normalization Notes

7.1 Third Normal Form Compliance

Each table satisfies 3NF because:

- All non-key attributes depend on the whole primary key and nothing but the primary key.
- No transitive dependencies exist between non-key columns.
- categories table eliminates repeating category strings across thousands of product rows.
- vendor profile data is separated from login credentials across vendors and users tables.

7.2 Intentional Denormalization

One deliberate denormalization exists for performance:

- order_items.vendor_id: Technically derivable via products.vendor_id JOIN, but stored directly to allow vendor sales queries with a single JOIN to order_items rather than two JOINs through products.
- orders.shipping_address (TEXT snapshot): The shipping address is copied from customers at checkout time. This is intentional — it preserves the historical delivery address even if the customer later updates their profile.

8. Document Revision History

Version	Date	Author	Changes
1.0	March 2026	Capstone Student	Initial release — full 12-table schema with all SQL, indexes, and relationships

Document: Database Design Document | Version 1.0 | CIS2987 Capstone | March 2026